EEET2601 Engineering Computing 1

Program Design

RMIT UNIVERSITY

Building Programs from Existing Information

- Often some—or all—of the solution can be developed from information that already exists or from the solution to another problem

- Basic program development process:
  - **Analysis phase**: Description of the problem's data requirements Focus
  - **Design phase**: Develop solution algorithm

Predefined Functions and Code Reuse

- A primary goal of software engineering is to write **error-free code**.

- *Code reuse*, reusing program fragments that have already been written and tested whenever possible, is one way to accomplish this goal.

  Stated more simply, "***Why reinvent the wheel?***"

Top-Down Design vs. Bottom-Up Design

Top-Down Design:

- Supported by *Structure/Procedure oriented* programming languages like **C**.
- Begin with high level design and ends with low level design or development.
- main() function is written first and all sub functions are called from main function. Then, sub functions are written based on the requirement

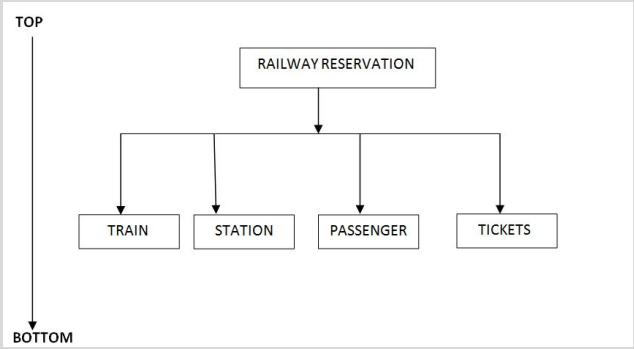
Bottom-Up Design:

- Supported by *Object oriented* programming languages like **C++** and **Java**
- Begin with low level design or development and ends with high level design.
- code is developed for modules and then these modules are integrated with main() function.

Nowadays, both approaches are combined together and followed in modern software design.

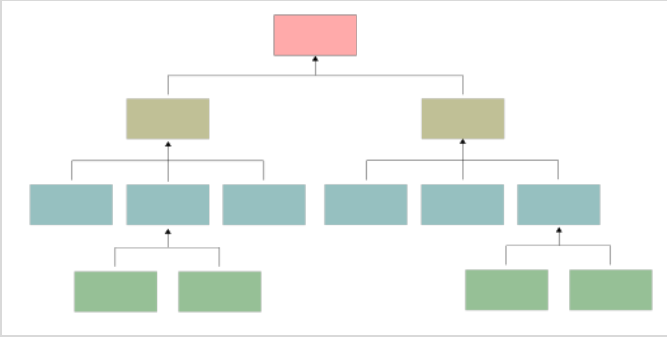
Top-Down Design vs. Bottom-Up Design

Top-Down design



A complex problem is divided into subproblems, a subproblem can also be divided into smaller subproblem

Bottom-Up design



Design small modules with dedicated functionality, then integrate them together

Structure Charts

Example: Calculate Average, ref: wikibooks

```c
#include <stdio.h>
void inputNums(float *a, float *b){
    printf("Enter two integer numbers: ");
    scanf("%f %f", a, b);
}

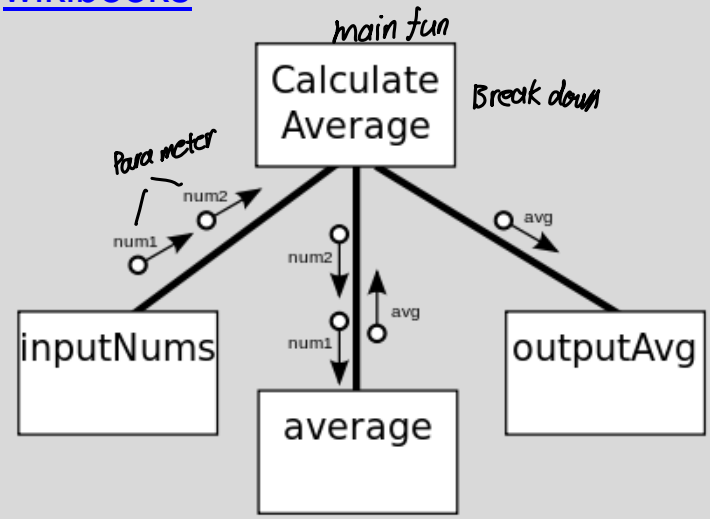
void outputAvg(float avg){
    printf("Average = %f: ", avg);
}

float average(float a, float b){
    return (a + b)/2;
}

int main(){
    float a, b, avg;
    inputNums(&a, &b);
    avg = average(a, b);
    outputAvg(avg);

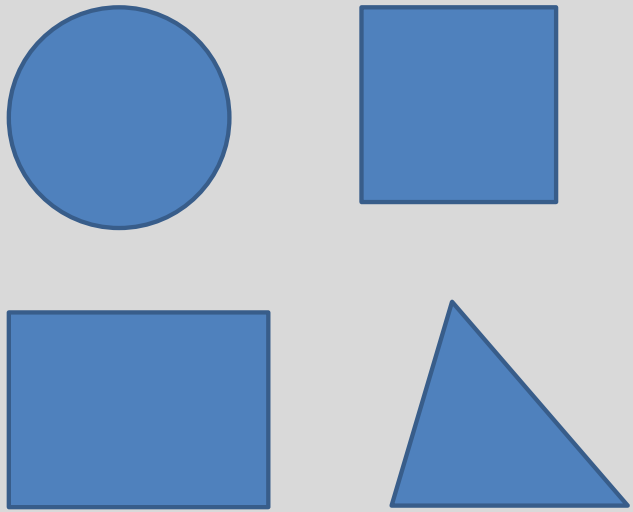
    return 0;
}

```



Example – Shape Area Calculator

- Create a program that can calculate the **area** of four basic shapes:
  - Circle
  - Square
  - Rectangle
  - Triangle

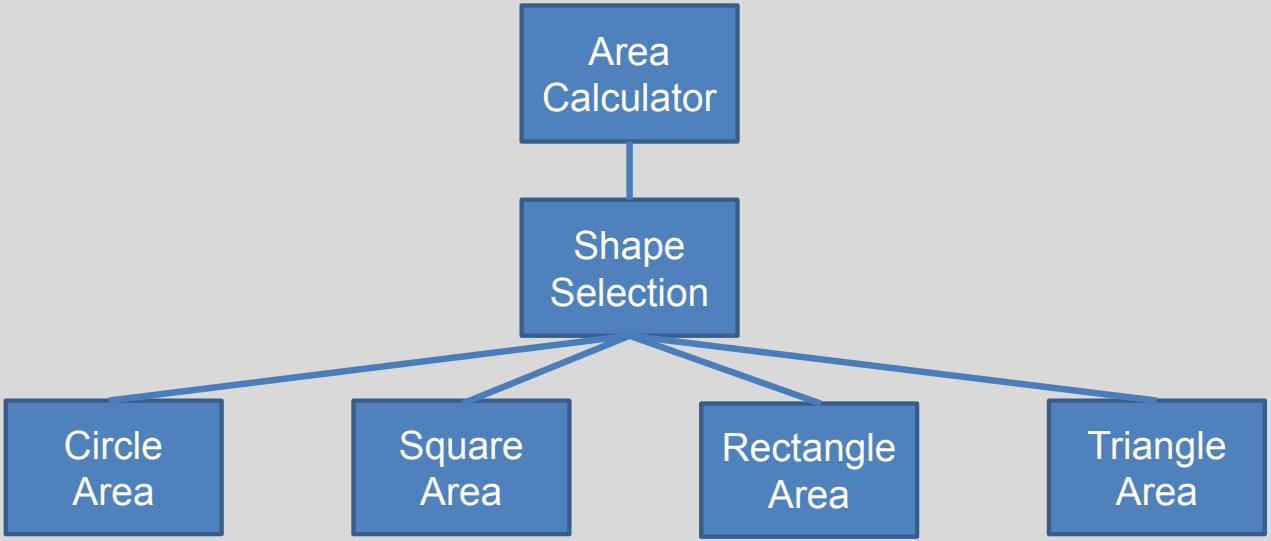


Example – Shape Area Calculator

- Circle Area: $A = \pi r^2$
- Square: $A = a^2$
- Rectangle: $A = a * b$
- Triangle: $A = \sqrt{s(s-a)(s-b)(s-c)}$

  with $s = (a + b + c)/2$

Example – Shape Area Calculator



Example – Shape Area Calculator

```c
#include <stdio.h>
#include <math.h>
#include <ctype.h>

void circle_area(){
    float radius;
    printf("Enter the radius: ");
    scanf("%f", &radius);
    printf("Area: %f", M_PI*radius*radius);
}

void square_area(){
    float side_lgth;
    printf("Enter the side lenght: ");
    scanf("%f", &side_lgth);
    printf("Area: %f", side_lgth*side_lgth);
}

void rctgl_area(){
    float width, length;
    printf("Enter the width and lenght: ");
    scanf("%f %f", &width, &length);
    printf("Area: %f", width*length);
}

void triagl_area(){
    float a, b, c, s;
    printf("Enter lenghts of three sides: ");
    scanf("%f %f %f", &a, &b, &c);
    s = (a + b + c)/ 2;
    printf("%f", s);
    printf("Area: %f", sqrt(s*(s-a)*(s-b)*(s-c)));
}

int main(){
    char shape;
    //Menu selection
    printf("Select the sharpe to calculate area\n"
           "\tC: circle\n"
           "\tS: square\n"
           "\tR: rectangle\n"
           "\tT: triangle\n");
    scanf("%c", &shape);
    shape = tolower(shape); //convert to lowercase

    //Call corresponding function
    switch(shape){
        case 'c':   circle_area();
                    break;
        case 's':   square_area();
                    break;
        case 'r':   rctgl_area();
                    break;
        case 't':   triagl_area();
                    break;
    }

    return 0;
}
```

RMIT Classification: Trusted

Example – Shape Area Calculator

shape_area.h

```
C: > Users > v12010 > Desktop > area_calculator > C shape_area.h
1   void circle_area();
2   void square_area();
3   void rctgl_area();
4   void triagl_area();
5   char shape_selection();
6
```

shape_area.c

```
C: > Users > v12010 > Desktop > area_calculator > C shape_area.c
1   #include <stdio.h>
2   #include <math.h>
3   #include <ctype.h>
4   #include "shape_area.h"
5
6   void circle_area(){
7       float radius;
8       printf("Enter the radius: ");
9       scanf("%f", &radius);
10      printf("Area: %f", M_PI*radius*radius);
11  }
12
13  void square_area(){
14      float side_lgth;
15      printf("Enter the side lenght: ");
16      scanf("%f", &side_lgth);
17      printf("Area: %f", side_lgth*side_lgth);
18  }
19
20  void rctgl_area(){
21      float width, length;
22      printf("Enter the width and lenght: ");
23      scanf("%f %f", &width, &length);
24      printf("Area: %f", width*length);
25  }
26
27  void triagl_area(){
28      float a, b, c, s;
29      printf("Enter lenghts of three sides: ");
30      scanf("%f %f %f", &a, &b, &c);
31      s = (a + b + c)/ 2;
32      printf("%f", s);
33      printf("Area: %f", sqrt(s*(s-a)*(s-b)*(s-c)));
34  }
35
36  char shape_selection(){
37      char shape;
38      //Menu selection
39      printf("Select the sharpe to calculate area\n"
40             "\tC: circle\n"
41             "\tS: square\n"
42             "\tR: rectangle\n"
43             "\tT: triangle\n");
44      scanf("%c", &shape);
45      shape = tolower(shape); //convert to lowercase
46      return shape;
47  }
48
```

main.c

```
C: > Users > v12010 > Desktop > area_calculator > C main.c
1   #include <stdio.h>
2   #include <math.h>
3   #include <ctype.h>
4   #include "shape_area.h"
5
6
7   int main(){
8       char shape = shape_selection();
9
10      //Call corresponding function
11      switch(shape){
12          case 'c':   circle_area();
13                      break;
14          case 's':   square_area();
15                      break;
16          case 'r':   rctgl_area();
17                      break;
18          case 't':   triagl_area();
19                      break;
20      }
21
22      return 0;
23  }
24
```

Reorganize to create a user-defined library

Example command to compile and run:

```
gcc main.c shape_area.c -lm -o area_calculator

./area_calculator
```

Practice

- Create a program that can do currency conversion among **USD**, **VND**, **AUD**

Example run as below:

```
Enter your currecy conversion question (e.g. "100USD to VND")
3.5usd to vnd
Answer: 3.50usd to vnd: 69402.00
```

References

1. Fresh2Refresh, C Programming Tutorial, 2020.
2. IncludeHelp, C/C++ Programming Language Tutorial, 2020.
3. Problem Solving and Program Design in C, 8th Edition. Jeri R. Hanly
4. C How to Program, 7th Edition. Paul Deitel